Lecture 11: Nonlinear systems, Quasi-Newton methods

Jamie Haddock

Table of contents

1 Newton for Nonlinear Systems

So far, we have considered only rootfinding problems defined for scalar functions of scalar variables.

1.1 Nonlinear systems

We will consider here the rootfinding problem defined by $\mathbf{f}: \mathbb{R}^n \to \mathbb{R}^n$.

> Definition: Rootfinding problem
>
> Given a continuous function $\mathbf{f}$ of a variable input $\mathbf{v}$, the **rootfinding problem** is to find a real input $\mathbf{r}$, called a **root** such that
>
> $$\mathbf{f}(\mathbf{r}) = \mathbf{0}$$
>
> or equivalently,
>
> $$\begin{aligned} f_1(r_1, \cdots, r_n) &= 0 \\ f_2(r_1, \cdots, r_n) &= 0 \\ &\vdots \\ f_n(r_1, \cdots, r_n) &= 0. \end{aligned}$$

1.2 Example

The steady state of interactions between the population $w(t)$ of a predator species and the population $h(t)$ of a prey species might be modeled as

$$ah - bhw = 0,$$
$$-cw + dwh = 0$$

for positive parameters a, b, c, d.

This fits into the form of the rootfinding problem by defining $\mathbf{x} = [h, w]$,

$$f_1(x_1, x_2) = ax_1 - bx_1x_2,$$

and

$$f_2(x_1, x_2) = -cx_2 + dx_1x_2.$$

1.3 Linear model

We use the same principle as in the scalar case: construct a simpler model of the exact nonlinear function.

We begin with a linar model based on the multidimensional Taylor series,

$$\mathbf{f}(\mathbf{x} + \mathbf{h}) = \mathbf{f}(\mathbf{x}) + \mathbf{J}(\mathbf{x})\mathbf{h} + O(\|\mathbf{h}\|^2),$$

where $\mathbf{J}$ is the **Jacobian matrix** of $\mathbf{f}$ defined as

$$\mathbf{J}(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & & \vdots \\ \frac{\partial f_n}{\partial x_1} & \frac{\partial f_n}{\partial x_2} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}.$$

1.4 Multidimensional Newton method

To derive Newton's method in this case, we assume $\mathbf{x}_k$ is reasonable approximation to a root $\mathbf{x}$ and define $\mathbf{h} = \mathbf{x} - \mathbf{x}_k$.

The linear model then becomes

$$\mathbf{f}(\mathbf{x}) \approx \mathbf{q}(\mathbf{x}) = \mathbf{f}(\mathbf{x}_k) + \mathbf{J}(\mathbf{x}_k)(\mathbf{x} - \mathbf{x}_k).$$

Now, we note that $\mathbf{f}(\mathbf{x}) = \mathbf{0}$ and require that $\mathbf{q}(\mathbf{x}_{k+1}) = \mathbf{0}$,

$$\mathbf{0} = \mathbf{f}(\mathbf{x}_k) + \mathbf{J}(\mathbf{x}_k)(\mathbf{x}_{k+1} - \mathbf{x}_k)$$

and thus,

$$\mathbf{x}_{k+1} = \mathbf{x}_k - [\mathbf{J}(\mathbf{x}_k)]^{-1}\,\mathbf{f}(\mathbf{x}_k).$$

The multidimensional Newton's method consists of three steps performed iteratively. Given $\mathbf{f}$ and starting value $\mathbf{x}_1$, for each $k = 1, 2, 3, \cdots$,

1. Compute $\mathbf{y}_k = \mathbf{f}(\mathbf{x}_k)$ and $\mathbf{A}_k = \mathbf{J}(\mathbf{x}_k)$.
2. Solve the linear system $\mathbf{A}_k\mathbf{s}_k = -\mathbf{y}_k$ for the **Newton step** $\mathbf{s}_k$.
3. Let $\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{s}_k$.

One can analyze the multidimensional Newton's method by generalizing the analysis we performed for the scalar Newton's method. This shows that the multidimensional Newton's method will be quadratically convergent in any vector norm, under suitable circumstances and *when the method converges at all.*

```
"""
    newtonsys(f,jac,x1[;maxiter,ftol,xtol])

Use Newton's method to find a root of a system of equations, starting from 'x1'.
The functions 'f' and 'jac' should return the residual vector and the Jacobian
matrix, respectively.  Returns the history of root estimates as a vector of
vectors.

The optional keyword parameters set the maximum number of iterations and the
stopping tolerance for values of 'f' and changes in 'x'.
"""
function newtonsys(f,jac,x1;maxiter=40,ftol=100*eps(),xtol=1000*eps())
    x = [float(x1)]
    y, J = f(x1), jac(x1)
    delx = Inf          # for initial pass below
    k = 1

    while (norm(delx) > xtol) && (norm(y) > ftol)
        delx = -(J\y)               # Newton step
        push!(x, x[k] + delx)       # append to history
        k += 1
        y, J = f(x[k]), jac(x[k])

        if k == maxiter
            @warn "Maximum number of iterations reached."
            break
        end
    end
    return x
end
```

```
newtonsys
```

```
function func(x)
    [ exp(x[2] - x[1]) - 2,
      x[1]*x[2] + x[3],
      x[2]*x[3] + x[1]^2 - x[2]
    ];
end;

function jac(x)
    [
        -exp(x[2]-x[1])  exp(x[2]-x[1])  0
        x[2]             x[1]            1
        2*x[1]           x[3]-1          x[2]
    ];
end;
```

```
x1 = BigFloat.([0,0,0])
  = eps(BigFloat)
x = newtonsys(func,jac,x1,xtol= ,ftol= );
```

Let's check the residual of the last result:

```
r = x[end]
@show residual = norm(func(r));
```

```
residual = norm(func(r)) = 0.0
```

```
logerr = [ Float64( log(norm(r-x[k])) ) for k in 1:length(x)-1 ]
[ logerr[k+1]/logerr[k] for k in 1:length(logerr)-1 ]
```

```
7-element Vector{Float64}:
 0.7937993447128696
 3.6959808854483027
 2.4326597889977153
 2.3110932374368063
 2.1325411149310054
 2.029767250340732
 2.0340010945857525
```

This suggests quadratic convergence!

2 Quasi-Newton methods

Newton's method is a very important algorithmic approach, but there are some drawbacks. Calculating the Jacobian in every iteration is very costly, and the fact that the method diverges from many starting points is very problematic. The **quasi-Newton methods** modify the method to try to overcome these challenges.

2.1 Jacobian by finite differences

In the scalar case, we developed the secant method by replacing the derivative $f'(x_k)$ by the difference quotient

$$\frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}.$$

If x_k converges to a root r, then this quotient converges to $f'(r)$.

In the system case, replacing the Jacobian is more complicated, due to the number of variables. The replacement will approximate the jth column,

$$\mathbf{J}(\mathbf{x})\mathbf{e}_j = \begin{bmatrix} \frac{\partial f_1}{\partial x_j} \\ \frac{\partial f_2}{\partial x_j} \\ \vdots \\ \frac{\partial f_n}{\partial x_j} \end{bmatrix} \approx \frac{\mathbf{f}(\mathbf{x} + \delta \mathbf{e}_j) - \mathbf{f}(\mathbf{x})}{\delta}, \qquad j = 1, \cdots, n.$$

```
"""
    fdjac(f,x0[,y0])

Compute a finite-difference approximation of the Jacobian matrix for 'f' at 'x0',
where 'y0 = f(x0)' may be given.
"""
function fdjac(f,x0,y0=f(x0))
      = sqrt(eps())*max(norm(x0),1)       # FD step size
    m,n = length(y0),length(x0)
    if n == 1
        J = (f(x0+ ) - y0) /
```

```
    else
        J = zeros(m,n)
        x = copy(x0)
        for j in 1:n
            x[j] +=
            J[:,j] = (f(x) - y0) /
            x[j] -=
        end
    end
    return J
end
```

`fdjac`

2.2 Broyden's update

Note that the finite-difference approximation to the Jacobian requires n additional function evaluations in each iteration, which may be too costly, especially since the function and Jacobian are supposed to change little as the iterates get close to the root.

Now, we'll seek a cheaper approximation to the Jacobian. The Newton iteration is defined by

$$\mathbf{A}_k \mathbf{s}_k = -\mathbf{y}_k$$

where $\mathbf{A}_k$ is the Jacobian.

We now assume the above linear system defines $\mathbf{s}_k$ but with $\mathbf{A}_k$ *approximating* the Jacobian. Once we use $\mathbf{s}_k$ to obtain $\mathbf{x}_{k+1}$, we need to update the approximate Jacobian to a new matrix $\mathbf{A}_{k+1}$. We'll assume

$$\mathbf{y}_{k+1} - \mathbf{y}_k = \mathbf{A}_{k+1}(\mathbf{x}_{k+1} - \mathbf{x}_k) = \mathbf{A}_{k+1}\mathbf{s}_k.$$

Thus, we require

$$\mathbf{A}_{k+1}\mathbf{s}_k = \mathbf{y}_{k+1} - \mathbf{y}_k.$$

This is not enough to uniquely define $\mathbf{A}_{k+1}$, but we'll also require that $\mathbf{A}_{k+1} - \mathbf{A}_k$ is rank-one.

> Definition: Broyden update formula
>
> We define
>
> $$\mathbf{A}_{k+1} = \mathbf{A}_k + \frac{1}{\mathbf{s}_k^\top \mathbf{s}_k}(\mathbf{y}_{k+1} - \mathbf{y}_k - \mathbf{A}_k\mathbf{s}_k)\mathbf{s}_k^\top.$$

If the method using this approximation fails to make improvement, one can reinitialize the $\mathbf{A}$ matrix by finite differences.

2.3 Levenberg's method

An important principle is that we can always check to see if a potential step reduces the backward error ($\|\mathbf{f}\|$) before accepting (or rejecting!) a potential step.

We will use here an alternative step defined by

$$(\mathbf{A}_k^\top \mathbf{A}_k + \lambda \mathbf{I})\mathbf{s}_k = -\mathbf{A}_k^\top \mathbf{f}_k.$$

Given $\mathbf{f}$, a starting value $\mathbf{x}_1$, and a scalar λ, for each $k = 1, 2, 3, \cdots$,

1. Compute $\mathbf{y}_k = \mathbf{f}(\mathbf{x}_k)$, and let $\mathbf{A}_k$ be an exact or approximate Jacobian matrix.
2. Solve the linear system above for $\mathbf{s}_k$.
3. Let $\hat{\mathbf{x}} = \mathbf{x}_k + \mathbf{s}_k$.
4. If the residual is reduced at $\hat{\mathbf{x}}$, then let $\mathbf{x}_{k+1} = \hat{\mathbf{x}}$.
5. Update λ and update $\mathbf{A}_k$ to $\mathbf{A}_{k+1}$.

When $\lambda = 0$, the system defining $\mathbf{s}_k$ is equivalent to the usual linear model (like used in Newton's method).

As $\lambda \to \infty$, the system approaches

$$\lambda \mathbf{s}_k = -\mathbf{A}_k^\top \mathbf{f}_k.$$

In this case, if $\mathbf{A}_k = \mathbf{J}(\mathbf{x}_k)$ then $\mathbf{s}_k$ is a steepest descent step for the objective $\|\mathbf{f}(\mathbf{x})\|^2$.

The λ parameter defines a smooth transition between the Newton step (with very rapid convergence near a root but potential divergence) and a gradient descent step (with guaranteed progress when far from a root).

```
"""
    levenberg(f,x1[;maxiter,ftol,xtol])

Use Levenberg's quasi-Newton iteration to find a root of the system 'f'
starting from 'x1'.  Returns the history of root estimates as a vector
of vectors.

The optional keyword parameters set the maximum number of iterations and
the stopping tolerance for values of 'f' and changes in 'x'.
"""
function levenberg(f,x1;maxiter=40,ftol=1e-12,xtol=1e-12)
    x = [float(x1)]
    yk = f(x1)
    k = 1; s = Inf;
    A = fdjac(f,x[k],yk)       # start with FD Jacobian
    jac_is_new = true

     = 10;
    while (norm(s) > xtol) && (norm(yk) > ftol)
        # Compute the proposed step.
        B = A'*A +  *I
        z = A'*yk
        s = -(B\z)

        x̂ = x[k] + s
        ŷ = f(x̂)

        # Do we accept the result?
        if norm(ŷ) < norm(yk)     # accept
             = /10              # get closer to Newton
            # Broyden update of the Jacobian
            A += (ŷ - yk - A*s)*(s'/norm(s)^2)
            jac_is_new = false

            push!(x,x̂)
            yk = ŷ
            k += 1
```

```
        else                        # don't accept
            # Get closer to gradient descent.
              = 4
            # Re-initialize the Jacobian if its out of date.
            if !jac_is_new
                A = fdjac(f,x[k],yk)
                jac_is_new = true
            end
        end
        if k==maxiter
            @warn "Maximum number of iterations reached."
            break
        end
    end
    return x
end
```

```
levenberg
```

```
x1 = [0.,0.,0.]
x = levenberg(func,x1)
```

```
12-element Vector{Vector{Float64}}:
 [0.0, 0.0, 0.0]
 [-0.08396946536317919, 0.07633587873004255, 0.0]
 [-0.42205075841965206, 0.21991260740534585, 0.012997569823167984]
 [-0.48610710938504953, 0.2138968287772044, 0.09771872586402451]
 [-0.45628390809556546, 0.24211047709245145, 0.10100440258901365]
 [-0.4556388336696561, 0.2347044354874538, 0.10854665717226099]
 [-0.4583961451067925, 0.2353095686241835, 0.10739828073307474]
 [-0.45804340381597397, 0.2351212406112955, 0.10768079583159754]
 [-0.45803332584412787, 0.23511390840121468, 0.10768998049540802]
 [-0.45803327880719313, 0.2351138986739345, 0.1076899925067127]
 [-0.4580332805601996, 0.23511389986307893, 0.107689990975689]
 [-0.458033280641234, 0.23511389991865286, 0.10768999090414474]
```

```
r = x[end]
println("backward error = $(norm(func(r)))")
```

```
backward error = 1.2687486282385731e-13
```

```
logerr = [ log( norm(x[k]-r) ) for k in 1:length(x)-1 ]
[ logerr[k+1]/logerr[k] for k in 1:length(logerr)-1 ]
```

```
10-element Vector{Float64}:
 1.348809392393256
 2.62941099249341
 1.4519728792043711
 1.3970235620907654
 1.289855471086
 1.2733028442786722
 1.458758678185501
 1.5234698610031927
```

```
1.1687771256502275
1.1579168224448166
```

Convergence is between linear and quadratic!